

Humanoid Embodied Agent

≡ Proposal	v2.1
------------	------

Members: Chen Sigen, Hong Zhuoyuan, Soni Samarth, Zhang Hengda

Project Overview & Motivation

This project aims to design and implement a **simplified humanoid (doll-like) embodied agent** that can interact with humans through **joint-level motion control** and **expressive** physical behaviours, translating human actions into physically grounded responses.

Traditionally, interaction systems rely on complex hardware or high-precision manipulation mapping. Instead, this project emphasises how **high-level interaction intents** (e.g., greeting, rejection, attention) can be grounded into **low-level joint movements** under physical constraints. The core problem addressed is how to **bridge** perception, decision-making, and actuation in a physically embodied system, moving away from static rule-based animation toward a **Generative Embodied Intelligence** paradigm.

System Architecture & Methodology

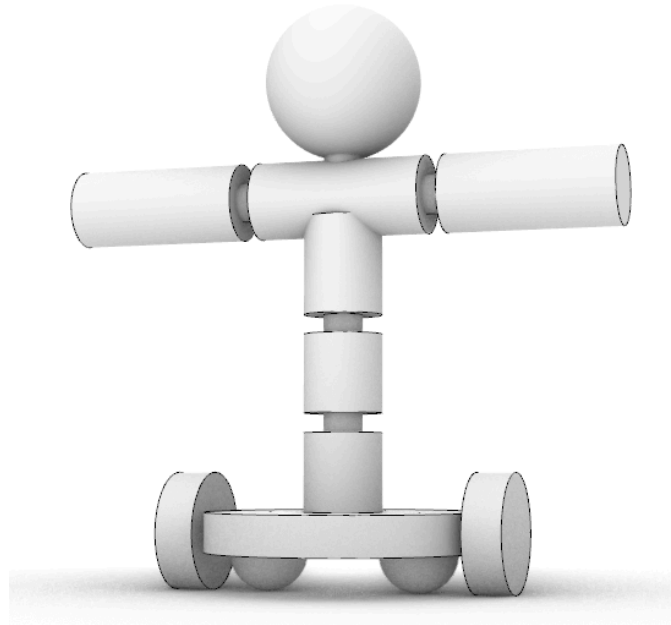
The system adopts a layered robotic architecture, separating perception, decision-making, and execution, while introducing a Vision-Language Model (VLM) to dynamically synthesise behaviours.

1. Hardware Design & Simulation Platform

To ensure feasibility within the timeframe and focus on algorithm integration, the project will adopt a simulation-first approach. The robot will be evaluated in a physics-based simulation environment (e.g., Gazebo/Coppeliasim).

1. **Custom Primitive URDF:** we will **not** use an off-the-shelf physical humanoid, nor will we tackle complex CAD from scratch. Instead, we will design our own simplified agent using a **custom primitive-based URDF** and test it purely in simulation (Gazebo for Physics and real simulation; RViz for visualisation of meta's).
 - a. We will approach in two stages: validate URDF kinematics in RViz first, then migrate to Gazebo for physics-based testing.

2. **Kinematic Structure:** To maximise expressive capacity while avoiding bipedal locomotion complexities, the agent will feature a simple **wheeled base** for planar movement, coupled with a highly expressive **9-DoF upper body** (Head: 2 DoF; Arms: 2 DoF $\times$ 2; Torso: 3 DoF including pitch/roll/yaw). We specifically added a 3-DoF torso (Pitch, Roll, Yaw) because it allows for much richer semantic grounding (e.g., leaning forward while greeting, swaying while idle) compared to just moving arms. This keeps the physical modelling lightweight while heavily supporting our focus on embodied interaction. For the current scope, the wheeled base and upper body controllers will be decoupled: upper-body behaviours will execute independently of base locomotion (latter one treated as a stretch-goal).
- a. Camera Placement: To start, A fixed environment camera will be used for perception to decouple visual input from head movement during execution



Sketch 3d model for Kinematic structure

2. Generative Behaviour Policy via Code-as-Policies (Learning-based Motion Generation)

To avoid the limitations of relying on predefined behaviours and rule-based animation systems, this project will introduce the **Code-as-Policies (CaP)** paradigm[1]. This approach upgrades the Vision-Language Model (VLM) from a simple intent classifier to a real-time, continuous behaviour generator.

1. **Prompt-driven Continuous Parameterisation:** To achieve dynamic behaviour synthesis, the system will not require the VLM to output discrete action labels

(e.g., `trigger("wave")`). Instead, the VLM will be provided with a set of low-level Python control APIs (e.g., `set_torso_pitch(angle, speed)`, `blend_arm_pose(target_pose, duration)`). Through rigorous prompt engineering, the system will prompt the VLM to generate continuous motion parameters in a zero-shot manner based on the social context derived from visual inputs (such as the human's distance, posture, and emotion). For instance, when executing a "greeting," if the human is far away, the VLM-generated code will assign a larger amplitude and higher velocity to the waving motion. If the human displays rejection, the VLM will generate a combination of parameters resulting in a backward torso lean and a slow arm retraction. This mechanism successfully leverages the contextual reasoning capabilities of large models to simulate the parameterisation of continuous motion primitives, achieving high generalisability and adaptability.

- a. **Intermediate Semantic Structure:** In the prompt, we will enforce a structured context object e.g. `{intent, social_distance, affect, confidence}`. This will then fill the prompt for API calls and also provide an interpretable intermediate representation for debugging and VLM-as-Judge evaluation.
2. **VLM Deployment Strategy:** To ensure the project's feasibility within the development timeline, the system will feature an API-agnostic ROS 2 client node. Initially, commercial APIs (e.g., GPT-4o) will be integrated to rapidly validate the "visual input to code generation" loop. If progress permits, the project will explore deploying a open-source multimodal large model (e.g., Qwen2-VL 7B) as an inference backend to further enhance the system's localisation and autonomy.
 - a. **Note:** We recognise we may face inference-latency constraints inherent to cloud APIs (maybe even 2–6 seconds per call), which is why we have a stretch-goal for a local inference backend. Lightweight multimodal models such as **LLaVA-1.5-7B**, **Qwen2-VL-7B**, **MiniCPM-V 2.6**, or **InternVL2-8B** can be deployed on a local GPU, reducing per-call latency to under 500ms. The API-agnostic client architecture means switching between cloud and local backends requires only a config change, allowing direct latency benchmarking between the two modes.
3. **Addressing Future Work: Latent Intent Embedding & Policy Networks:** While prompt-driven continuous parameterisation is highly effective, a more native architecture would bypass discrete language and code layers entirely by directly extracting high-dimensional latent intent embeddings (z) from the VLM's vision-language encoder's intermediate layers. In future iterations, a

dedicated behaviour policy network (π) could be introduced. This network would take the latent vector z as input to directly output low-level joint trajectories end-to-end, allowing the system to capture highly nuanced interaction contexts and system uncertainties. However, considering that training such a policy network requires massive datasets and long convergence cycles, simulating continuous parameterisation via prompts serves as the optimal balance between academic exploration and engineering feasibility for the current development cycle.

4. **Our contribution:** The focus will be on the system-level framework (not fine-tune VLM weights): a robotics-domain API specification, a structured prompt schema (JSON/YAML) that enforces contextually parameterised code output, and a safety-constrained execution sandbox: together enabling a general-purpose VLM to function as a reliable embodied behaviour generator without task-specific training.

3. Low-level Control & Hierarchical Execution

Once the high-level generative policy is established, the execution layer must resolve the inherent conflict between VLM inference latency and the high-frequency control required for physical robots. This system will adopt a classic **Hierarchical Control Architecture**, thoroughly decoupling high-level semantic planning from low-level smooth execution.

1. **High-Low Frequency Decoupling:** The VLM node operates at a low frequency (approximately 0.5Hz - 1Hz), dedicating its computational resources to processing visual perception and generating Python control scripts that contain target parameters (waypoints and durations). Once generated, these scripts are dispatched to the high-frequency execution layer.
 - a. **Latency masking:** We recognise the robot must continue to act during the few seconds of inference. While the VLM processes the next frame, the robot maintains its current behaviour or executes a default idle motion, ensuring no frozen or jerky pauses during interaction.
2. **ROS 2 Joint Trajectory Execution:** The underlying layer operates on the ROS 2 `ros2_control` framework. Upon receiving the sparse target parameters generated by the VLM, the `joint_trajectory_controller` utilises spline interpolation at a frequency of 100Hz to automatically generate smooth joint trajectories. This architecture not only masks the VLM's inference latency (requiring only 1-2 seconds of initial planning for responsive actions) but also ensures absolute

smoothness in motion execution (minimising jerk), completely avoiding the mechanical jitter associated with direct control.

3. **Safety Sandbox & Constraint Enforcement:** To prevent destructive commands resulting from potential VLM "hallucinations," the Python code output by the VLM will be parsed and executed within a restricted sandbox environment. Any action that attempts to drive the joints beyond the physical limits defined in the URDF, or attempts to execute when the Center of Mass (CoM) severely deviates from the support polygon, will be intercepted by the underlying middleware, immediately triggering a safe recovery mechanism.
 - a. Additionally, the sandbox will perform a geometric self-collision check before execution, rejecting trajectories where arm or torso links would intersect based on the URDF bounding volumes.
4. **Fallback/degradation policy:** For reliability concerns, we will have a three-tier fallback:
 - a. Retry VLM inference once on API timeout or invalid code output,
 - b. Fall back to the most recently valid executed behaviour if retry fails,
 - c. Return to idle pose if no prior valid state exists.

Automated Evaluation Framework

To rigorously validate the system beyond qualitative demonstrations, a structured evaluation framework will be introduced. This pipeline operates entirely within the simulation:

1. **Algorithmic Reliability:** We will measure the VLM's **Execution Success Rate** (percentage of syntactically valid code generated) and **Safety Constraint Adherence** (percentage of trajectories within URDF joint limits)
2. **Physical / Motion Level:** To address motion smoothness and stability, we will record ROS 2 `/joint_states`. We will quantify smoothness by calculating the **Average Joint Jerk** ($d^3\theta/dt^3$). For stability, we will track the **Center of Mass (CoM) projection** to ensure dynamic balance within the wheeled base's support polygon during upper-body movements.
3. **Semantic Interaction Performance:** To measure interaction performance metrics without extensive user studies, we will utilise a **VLM-as-a-Judge** methodology. An independent VLM evaluator will compare the input human visual cue with a simulation screenshot of the robot's physical response to

calculate an *Interaction Appropriateness Score (Pass/Fail)*, validating behaviour correctness and contextual alignment. The judge receives:

- a. the input camera image of the human,
 - b. a screenshot of the robot's simulated response, and
 - c. a prompt asking it to assess whether the robot's physical behaviour is socially appropriate given the human's posture and context, returning a binary Appropriateness score with a brief rationale.
4. Fallback Activation Rate (percentage of inference cycles that triggered any fallback tier) will be recorded to quantify system robustness under failure conditions.
 5. FIXED TEST SCENARIOS: We will define 8 canonical interaction scenarios (e.g., person waves, person crosses arms, person leans forward, person walks away, person crouches) each with a documented expected robot response. All evaluation metrics will be computed across this fixed scenario set. With this, our evaluations will function as a benchmark for future updates and improvements.
 6. Rule-based baseline: We will implement both: a rule-based baseline (VLM → single intent label → fixed behaviour) and our CaP approach on the same 8 scenarios. We will then compare our generative CaP system against the rule-based baseline (fixed behaviour lookup based on discrete intent classes) on the canonical scenario set using the VLM-as-Judge metric (Interaction Appropriateness Score).

Task Distribution

To ensure efficient parallel development, we have structured our team's division of labor into four distinct, modular roles:

1. **Simulation & URDF.** Responsible for building the custom 9-DoF primitive URDF, configuring Gazebo physics, and setting up simulated camera sensors.
2. **Embodied Intelligence.** Focuses on the Vision-Language Model integration, developing the API client, and engineering the 'Code-as-Policies' prompt structure for reliable code generation.
3. **ROS 2 Control & Middleware.** Handles `ros2_control` configuration, implements the underlying Python action APIs called by the VLM, and enforces joint safety limits during execution.

4. **Evaluation & Metrics.** Develops the automated testing scripts to calculate Joint Jerk, track CoM stability, and implement the VLM-as-a-Judge semantic evaluation pipeline.

Demonstration of Course Mastery

This project demonstrates mastery of the robotic systems curriculum by seamlessly integrating:

- Embodied intelligence principles
- ROS 2 system architecture and node-based communication.
- Kinematic modelling and joint-level trajectory execution.
- Advanced embodied AI paradigms (VLM-driven generative policies) with classical control loops.
- Rigorous quantitative evaluation encompassing physical dynamics (CoM, Jerk) and semantic interaction.

References

[1] J. Liang *et al.*, "Code as policies: Language model programs for embodied control," *arXiv preprint arXiv:2209.07753*, 2022. [Online]. Available: <https://arxiv.org/pdf/2209.07753>

[2] AI Usage: We have taken the help of AI models such as Claude, Gemini, ChatGPT to help us find potential blind spots in our design and thought process and to streamline our ideas into a structured Product Requirements Document.